QueryAPIGate HOW-TO GUIDE 23 · OPERATE

Put QueryAPIGate behind a reverse proxy with real TLS

Current as of QueryAPIGate **v0.49.0** · PDF made 4 November 2026

Latest version: anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?g=23-put-it-behind-a-reverse-proxy

Time: 20 minutes. **You'll end up with:** QueryAPIGate reachable only over HTTPS, through Caddy or nginx, with rate limits and IP allowlists seeing each caller's real address - and live events and large exports streaming through the proxy rather than piling up in it.

QueryAPIGate speaks plain HTTP and doesn't terminate TLS itself. API keys travel in a header on every request, so anything reachable beyond `localhost` needs HTTPS in front of it.

Both configurations below were run against a real server: Caddy with a locally issued certificate on `https://localhost:8443`, nginx with a self-signed one on `https://localhost:9443`. For a public host, use your real domain and ports 80/443 as shown.

The three things the proxy setup must get right

1. **Only the proxy is reachable.** Bind QueryAPIGate to `127.0.0.1` (the default for `queryapigate serve`) or keep it on a private Docker network with no published port.
2. **QueryAPIGate knows a proxy is there:** `QUERYAPIGATE_TRUST_PROXY=1` (the number of proxies in front of it). Then it takes the client address, scheme and host from `X-Forwarded-For`, `-Proto` and `-Host`.
3. **Streaming isn't buffered:** live events (`/events`) and streamed exports (`?stream=true`) must reach the client as they're produced.

Option A: Caddy (simplest - automatic certificates)

caddyfile

```
# Caddyfile
api.example.com {
    reverse_proxy 127.0.0.1:5000
}
```

That's all. Caddy obtains and renews a certificate for `api.example.com` (DNS must point at the host, and ports 80 and 443 must be reachable for the challenge), redirects HTTP to HTTPS, sets the `X-Forwarded-*` headers - replacing any a client sent - and streams responses without buffering. Verified: `http://` answered `308` to `https://`, and a live event arrived through Caddy the moment a query ran.

Option B: nginx

```nginx
server {
    listen 443 ssl;
    server_name api.example.com;
    ssl_certificate     /etc/letsencrypt/live/api.example.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/api.example.com/privkey.pem;

    location = /metrics {
        allow 10.0.0.0/8;   # your Prometheus
        deny all;
        proxy_pass http://127.0.0.1:5000;
    }

    location / {
        proxy_pass http://127.0.0.1:5000;
        proxy_set_header Host              $host;
        proxy_set_header X-Forwarded-For   $remote_addr;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_set_header X-Forwarded-Host  $host;
    }

    # Live events and streamed exports: pass each chunk on as it arrives
    location ~ ^/(events|q/|execute_sql) {
        proxy_pass http://127.0.0.1:5000;
        proxy_set_header Host              $host;
        proxy_set_header X-Forwarded-For   $remote_addr;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_set_header X-Forwarded-Host  $host;
        proxy_buffering off;
        proxy_read_timeout 1h;
    }
}

server {
    listen 80;
    server_name api.example.com;
    return 301 https://$host$request_uri;
}
```

Notes on the choices:

- `X-Forwarded-For $remote_addr` **replaces** whatever the client sent, rather than appending to it. With `$proxy_add_x_forwarded_for` and one trusted hop, QueryAPIGate still takes the right (last) address - but replacing is simpler to reason about.
- `proxy_buffering off` on the streaming routes: without it nginx collects a live event or an export chunk before passing it on. `proxy_read_timeout 1h` lets a long export finish; live events send a keepalive every 15 seconds, so they'd survive nginx's 60-second default anyway.
- Certificates from certbot or your own CA.

Start QueryAPIGate for it

```shell
QUERYAPIGATE_API_KEY=... QUERYAPIGATE_TRUST_PROXY=1 \
  gunicorn --bind 127.0.0.1:5000 --workers 1 --worker-class gthread --threads 8 --timeout 120 \
  "queryapigate.app:create_app()"
```

(`queryapigate serve` is Flask's development server - fine for trying this out, not for production. The Docker image runs gunicorn already; see Deploy with Docker.)

Check it

```shell
curl https://api.example.com/health
# {"status": "ok", "version": "...", ...}
```

- **Generated URLs use HTTPS and your domain.** The Postman export's `baseUrl` came out as `https://localhost:8443` through Caddy and `https://localhost:9443` through nginx - QueryAPIGate builds it from the forwarded scheme and host.
- **Spoofed addresses don't work.** A key with `"allowed_ips": ["6.6.6.6"]`, called through either proxy with `X-Forwarded-For: 6.6.6.6` in the request, got `401`: the proxy replaced the header with the real address.
- **The real address does.** The same key with the caller's actual address listed got `200`. (On one machine, curl reached the proxy over IPv6 - the address was `::1`, not `127.0.0.1`. Check which address your callers really have.)
- **Streaming works:** a 20,000-row `?stream=true` CSV export arrived complete through nginx, and a live event arrived over `/events` as soon as a query ran.
- **`/metrics` is private:** `403` from outside the allowed range. It's public by design on QueryAPIGate itself (no key), so the proxy is the place to restrict it.

Headers: what's left to the proxy

QueryAPIGate sets its own security headers on every response (since 0.23) - `nosniff`, a CSP that runs nothing on API responses, `no-referrer`, `Cache-Control: no-store` - so the proxy doesn't need to add them (the list). What's left is HTTPS itself and, once every client reaches the server over HTTPS, HSTS. Either set it at the proxy, or let QueryAPIGate send it:

```shell
QUERYAPIGATE_TRUST_PROXY=1 QUERYAPIGATE_HSTS_MAX_AGE=31536000 ...
curl -sD - -o /dev/null http://127.0.0.1:5000/health -H 'X-Forwarded-Proto: https' | grep Strict
# Strict-Transport-Security: max-age=31536000
```

It goes only on requests that arrived over HTTPS - the forwarded scheme, behind a trusted proxy - never on plain HTTP, where it would lock browsers out of a server without HTTPS. Start with a short `max-age` (an hour, `3600`) and raise it once HTTPS is settled: browsers remember it for that long.

Mistakes this avoids

MISTAKE	WHAT HAPPENS
No `QUERYAPIGATE_TRUST_PROXY` behind a proxy	every caller appears to be the proxy: one shared rate-limit bucket, `allowed_ips` can't tell callers apart
`QUERYAPIGATE_TRUST_PROXY` with **no** proxy	callers choose their own address with `X-Forwarded-For` - rate limits and `allowed_ips` become useless
Proxy count too low (e.g. CDN + nginx, but set to `1`)	the CDN's address is taken as the client
QueryAPIGate also listening publicly	clients can bypass the proxy - and TLS - entirely
`proxy_buffering` on for `/events`	live events arrive late, in batches

The other two processes

`queryapigate mcp` (port 5001) and `queryapigate events` (port 5002) are separate servers; proxy them the same way if clients reach them from outside. `queryapigate events` reads `QUERYAPIGATE_TRUST_PROXY`; the MCP server doesn't - it sees the proxy's address for every caller (see Restrict a key to specific source IPs).

Next steps

- Deploy with Docker for real - the same proxy, as a Compose service.
- Allow a browser-based frontend to call the API.
- Rate-limit a key, or the whole server.